Les Fadas de l'IA 🫒

Votre agent IA personnel, sur votre serveur, opérationnel en moins de 2h.

Hostinger KVM2 | Ubuntu 24.04 LTS | pi-mom | Slack

Avant de commencer

Prérequis — 4 comptes en 5 min

- **Compte Google ou email** — Google accélère le login partout
- **Hostinger** — login Google, moins d’1 minute
- **GitHub** — login Google, 2 minutes
- **Slack** — login Google, 2 minutes
- **Abonnement LLM** — ChatGPT Plus, Gemini Advanced *(au choix)* ou GitHub Copilot Individual à $10/mois *(annulable avant 30 jours)*

💡 Pas d'abonnement LLM ? GitHub Copilot Individual fait l'affaire — 3 minutes pour s'inscrire

Suivi live — même réseau Wi-Fi que la salle

L’histoire

Novembre 2025 — Une nuit à Vienne

Peter Steinberger, développeur autrichien. Après 3 ans de voyage, il revient au code avec une idée simple :

Avant — Vous demandez → l'IA répond → fin.

↓

Son idée — Vous demandez → l'IA **fait**.

📱 WhatsApp ---message--- 🧠 Claude ---action--- 💻 Serveur

1 heure de code. L'IA exécute des commandes, crée des fichiers, cherche sur le web.
Elle agit.

💬 Vous : "Résume le PDF dans mon dossier Downloads"
🤖 Agent : lit le fichier, écrit le résumé, vous l'envoie

💬 Vous : "Crée-moi un site et mets-le en ligne"
🤖 Agent : code, déploie, envoie le lien

💬 Vous : "Rappelle-moi le dentiste demain à 9h"
🤖 Agent : programme le rappel, l'envoie lui-même

Les grandes boîtes n'avaient pas fait ça. Ni OpenAI. Ni Google. Ni Anthropic.

285 000 ⭐

Parmi les repos les plus ★ de l’histoire de GitHub

pics médiatisés ~4–5 mars 2026 (~250k★ dépassent React ~3 mars ; ~285k★ peu après) — le repo existe depuis ~nov. 2025

(~369k★ au 5 mai 2026 sur github.com/openclaw/openclaw — et ça continue)

*"OpenClaw est probablement la release logicielle la plus importante, you know, **probably ever**. Linux a mis 30 ans. OpenClaw : 3 semaines."*

Jensen Huang, CEO NVIDIA — reprise très médiatisée (ex. CNBC, 5 mars) + keynote GTC San Jose (~17 mars 2026, Reuters)

La timeline

Nov 2025	**Clawdbot** 🦞 — 1 heure, 1 développeur autrichien, 1 idée folle
Jan 27, 2026	**Clawdbot** devient **Moltbot** 🦞 — 1er renommage (pression trademark **Anthropic** sur la proximité avec « Claude »)
Jan 28, 2026	**Moltbook** — réseau social pour agents (Matt Schlicht). Viral ; ordre de grandeur **~1,5M+** « agents » enregistrés très vite (chiffres variables selon sources / fuites).
Jan 30, 2026	**Moltbot** devient **OpenClaw** 🦞 — nom final du projet de Steinberger
Fév 14, 2026	Steinberger rejoint OpenAI. Le projet passe en **fondation open-source**.
~3–5 mars 2026	OpenClaw grimpe très vite aux sommets GitHub★ (souvent cité comme dépassant **React** — pics ~250k★ puis ~285k★ selon le jour).
~17 mars 2026	**Jensen Huang** met OpenClaw au centre au **GTC** — *«every company … needs an OpenClaw strategy»*, comparaisons avec Linux/Windows...
Mar 10, 2026	**Meta** rachète **Moltbook** — la plateforme rejoint **Meta Superintelligence Labs** (MSL). Matt Schlicht & Ben Parr rejoignent Meta. Annonce presse (Axios, TechCrunch …) — montant non divulgué.
Mar 16, 2026	NVIDIA annonce **NemoClaw** pour la communauté OpenClaw (couverture presse ~16 mars). Écosystème : l'agent minimal **Pi** (**Mario Zechner**) est souvent décrit comme le moteur type des déploiements OpenClaw ; Nvidia et la presse associent aussi **Huang × Steinberger** pour la couche sécurité / enterprise.

Et dans 1h30

vous aurez votre propre instance.

Privée. Sur votre serveur. Qui vous appartient.

@LeJ — La démonstration

Voilà ce que vous aurez

Tout le monde est dans le workspace Slack **Les Fadas de l'IA** → canal `#agent`

Démo 1
Crée une blague en bash et exécute-la

Démo 2
Cherche la météo à Marseille

Démo 3
Écris un haiku sur les agents IA

Démo 4 🛡️
Demande risquée → l'agent demande confirmation

"C'est exactement ça que vous allez avoir. Commençons."

Installation

Étape 1 — Commander le VPS

Hostinger → VPS → **KVM2** — ~10€/mois *(ou lien + QR ci-dessous)*

OS : **Ubuntu 24.04 LTS** — Région : Europe

Valider → le VPS se provisionne (5-15 min selon la charge)

Panier direct — KVM2 · 1 mois

Ouvrir Hostinger KVM2

Lien affiliation (même prix affiché chez Hostinger). URL complète affichée sur le QR — suffisamment grand pour scanner depuis le fond de la salle.

Étape 2 — Slack workspace + App

Workspace

- slack.com → "Create a workspace"
- Nom = nom de votre agent
- Créer le canal `#agent`

Slack App

- api.slack.com/apps → From Scratch
- Socket Mode → **ON**
- App Token `connections:write` → copier `xapp-…`
- Bot Scopes → Install → copier `xoxb-…`

```
MOM_SLACK_APP_TOKEN = xapp-...
MOM_SLACK_BOT_TOKEN = xoxb-...
```

Bot Token Scopes à cocher (13 scopes) :
`app_mentions:read` · `chat:write` · `files:write` · `im:write` · `assistant:write` · `channels:history` · `groups:history` · `im:history` · `channels:read` · `files:read` · `im:read` · `groups:read` · `users:read`

⚠️ Après avoir ajouté `users:read`, cliquez "Reinstall to Workspace" en haut de la page.

Étape 3 — Aujourd’hui, on va jusqu’où ?

But de la séance : votre **agent répond dans Slack** sur le VPS, workspace propre.
Rien d’autre n’est obligatoire pour dire « l’atelier est réussi ».

- **Script 1** — Clé SSH (PC) + compte Linux + pare-feu. *Une fois.*
- **Script 2** — Node, build **pi-mom**, `~/agent-workspace`, **lingering** (actif sans session SSH).
- **Script 3** — Slack, LLM, nom, `SYSTEM.md`, dossiers `projects/` `missions/` `tools/`, **systemd**.

Pas de GitHub dans le script 3 ?

D’abord **voir l’agent vivre**. Git reste **local sur le VPS** — pas besoin de compte GitHub pour la démo.

Premiers réflexes terminal

Avant de plonger : 3 choses utiles à savoir si c'est votre première fois.

👀 **Lisez avant d'appuyer sur Entrée**

Surtout pour les commandes qu'on copie-colle. Une coquille dans une URL et le script ne charge pas.

🙈 **Quand vous tapez un mot de passe, RIEN ne s'affiche**

Pas d'étoiles, pas de points. C'est normal, c'est sécurisé. Tapez et appuyez sur Entrée.

📋 **Copier-coller dans le terminal**

Mac : `Cmd+C` / `Cmd+V`. Windows/Linux : `Ctrl+Shift+V` dans le terminal (pas `Ctrl+V`). Clic droit → Coller marche aussi partout.

Avant le serveur — générer une clé SSH

Même nom partout : `MON_AGENT` → identifiant court *sans espaces* (ex. `marie`, `bot01`).

Nom du fichier sous `~/.ssh/`, même nom en `ssh -i …` et user Linux au script 1.

À faire **sur votre PC** (terminal local), pas sur le VPS.

Mac

```
ssh-keygen -t ed25519 -f ~/.ssh/MON_AGENT -N "" && cat ~/.ssh/MON_AGENT.pub | pbcopy && echo "✅ Clé copiée"
```

Linux

```
ssh-keygen -t ed25519 -f ~/.ssh/MON_AGENT -N "" && cat ~/.ssh/MON_AGENT.pub | xclip -selection clipboard && echo "✅ Cl
```

Windows (PowerShell)

```
ssh-keygen -t ed25519 -f "$env:USERPROFILE\.ssh\MON_AGENT" -N '""'
Get-Content "$env:USERPROFILE\.ssh\MON_AGENT.pub" | Set-Clipboard
Write-Host "Copié dans le presse-papiers"
```

Pas de `xclip` sur Linux ? `sudo apt install xclip` ou ouvrez `cat ~/.ssh/MON_AGENT.pub` et copiez la ligne à la main.

Étape 4 — SSH + Script 1

Prérequis : clé créée (slide précédente). IP + mot de passe root = e-mail Hostinger.

1) Connexion au VPS — remplacez `MON_AGENT` (même nom que pour la clé) et `VOTRE_IP` :

```
ssh root@VOTRE_IP
```

- Première fois : si SSH demande la signature du serveur → tapez `yes` puis Entrée.
- Mot de passe root : rien ne s'affiche quand vous tapez, c'est normal.
- Pas de clé encore ? Générez-la sur la slide précédente, puis revenez ici.

2) Sur le VPS (invite `root@…#`), lancer le script 1 :

```
curl -fsSL https://raw.githubusercontent.com/clawassistantgf-dev/workshop-pi-mom/main/01-secure.sh -o "$HOME/ws-pi-mom-
```

- **Nom** → votre futur user Linux (cohérent avec `MON_AGENT` si vous voulez vous simplifier la vie).
- **Mot de passe** (double saisie, invisible).
- **Clé publique** — collez celle du presse-papiers (slide d'avant).

Si `curl: (23) Failure writing` : `df -h`, puis essayez `-o /root/ws-pi-mom-01-secure.sh` à la place de `"$HOME/…"`.

Étape 5 — Script 2 : Installation

```
curl -fsSL https://raw.githubusercontent.com/clawassistantgf-dev/workshop-pi-mom/main/02-install.sh -o /tmp/ws-pi-mom-0
```

⏳ ~8-16 minutes (npm workspaces + compilation `pi-tui` puis `coding-agent`). Le script est **idempotent** — relançable sans danger. Le hook `prepare` **husky** du monorepo est neutralisé sur le VPS (sinon erreur 127) ; pareil : `HUSKY=0`.

📦 Node.js 20 LTS

via NodeSource — runtime de pi

🐳 Docker (optionnel)

seulement si `--with-docker` ou `INSTALL_DOCKER=1` — pi-mom ne l'utilise pas

🤖 pi

`@mariozechner/pi-coding-agent`

💬 pi-mom version Les Fadas

fork de `pi-mom`, build local depuis sources

Crée aussi `~/agent-workspace` avec un `.gitignore` qui protège déjà `.env`.
Active le **lingering systemd** pour que votre agent tourne 24/7 même quand vous êtes déconnecté.

Pendant que ça installe…

Le script tourne ~10 min. Bon moment pour expliquer ce qu'il fait — et la suite contient une bonne nouvelle.

🖥️ **Agent sur le serveur (sans sandbox Docker)**
`pi-mom` tourne avec **votre utilisateur Unix** : même pouvoir qu'un shell sur le VPS. Soyez conscients des risques — pas de conteneur qui isole l'agent.

⏰ **Lingering systemd, c'est quoi ?**
Par défaut, vos services s'arrêtent quand vous fermez SSH. Lingering = ils tournent même sans vous. Indispensable pour un agent toujours actif.

🔄 **Si ça plante en cours de route ?**
Le script est idempotent. Relancez-le, il reprend où il en était.

⚠️ Si vous avez installé Docker (`--with-docker`) : **reconnectez-vous** ou `newgrp docker` pour le groupe docker. Sinon, ignorez ce message.

Pi-mom Les Fadas — ce que ce build contient déjà 🦞

Vous installez le fork **clawassistantgf-dev/pi-mono** — build local au script 2. Les points ci-dessous ne sont **pas** une feuille de route « à venir » : ils sont **déjà livrés** dans la version que vous compilez aujourd'hui.

⏳ Sabliers visibles dans Slack

Pendant que l'agent travaille, vous voyez l'activité — moins de « est-ce qu'il a planté ? ».

🚦 File d'attente par fil de discussion

Un flux ordonné par conversation : les messages sont traités proprement, dans l'ordre.

🧠 Mémoire persistante

Historique conservé entre les sessions pour enchaîner sur ce qui a déjà été fait.

🔄 Rechargement de SYSTEM.md

Personnalité et règles : le fork est conçu pour intégrer les mises à jour de configuration sans friction manuelle.

Sources : `github.com/clawassistantgf-dev/pi-mono` (fork) · upstream `github.com/badlogic/pi-mono`.

Étape 6 — pi /login (OAuth)

Votre agent rencontre son cerveau — flux commun à tous les providers.

```
pi
```

Ensuite taper `/login` puis choisir le fournisseur (ChatGPT, Gemini, **Copilot**, …).

Résumé : URL du terminal → navigateur PC → redirection **localhost** dans la barre → recoller dans la session SSH où `pi` tourne.

Copier l'URL complète affichée par pi (sans la tronquer).

Navigateur **sur votre PC** → connexion au provider → **Autoriser**.

Dans la barre d'adresse : copier `http://127.0.0.1:…` ou `http://localhost:…`.

Coller dans le terminal SSH où pi attend l'URL. Puis éventuellement `Ctrl+C`.

Pas de clé API sur le VPS : OAuth + votre abonnement fournisseur.

Étape 6 — Copilot · modèle · ensuite

GitHub Copilot (`github.com`)

- **a)** *GitHub Enterprise URL* : **vide** + **Entrée** pour le compte public standard.
- **b)** Copier **toute** l'URL OAuth du terminal (comme pour les autres logins).
- **c)** Après GitHub, reprendre l'URL **localhost / 127.0.0.1** du navigateur et la coller dans pi.

Modèle & test

- `Ctrl+L` : liste / choix du modèle (voir raccourcis en bas de pi).
- Une **question courte** pour valider que ça répond.

→ **Étape 7 — Script 3** : Slack, `.keys`, systemd… Vérifie `~/.pi/agent/auth.json` après `/login`.

Étape 7 — Script 3 : Configuration

```
curl -fsSL https://raw.githubusercontent.com/clawassistantgf-dev/workshop-pi-mom/main/03-configure.sh -o /tmp/ws-pi-mom
```

Après avoir réussi **pi** + `/login`, le script 3 demande le token Slack **APP**, le token Slack **BOT**, le fournisseur / modèle LLM et le **nom** de l'agent. **Aucune** question GitHub — c'est volontaire (voir `docs/MISSIONS.md` après l'atelier).

- .keys écrit
- SYSTEM.md créé
- structure projects/ missions/ tools/
- git local + hook validate
- systemd actif (pi-mom direct)

Ce soir ou plus tard — GitHub (optionnel)

Ce n'est pas une « étape » du même genre que VPS ou Slack : c'est un **bonus** quand vous voudrez **sauvegarder** ou **partager** votre workspace hors du serveur.

- Le script **3** ne pose **aucune** question GitHub — **normal**, ce n'est pas le critère de réussite du jour.
- Vous créerez un repo (souvent **privé**), une **clé SSH**, un remote `git@github.com:…`, puis `git push` — guidé dans `docs/MISSIONS.md` avec LE J.
- **Pas de push HTTPS** sur ce parcours ; `.keys` et secrets **jamais** commités *(déjà exclus par `.gitignore`)*.

💡 Si vous n'avez **jamais** utilisé Git : ce n'est **pas** un prérequis pour dire bonjour à votre agent dans Slack — on le fait ensemble **après**, au calme.

Premier contact 🎉

Ouvrez Slack

```
@[VOTRE_AGENT] Bonjour !
Qui es-tu et qu'est-ce que tu sais faire ?
```

Laissez tourner. Explorez. Demandez-lui n'importe quoi.

Sécurité

3 règles, pas plus

🔒 .env ne quitte jamais le serveur

Il est dans `.gitignore` dès le Script 2. Si quelqu'un y accède : changez les tokens immédiatement.

🧠 Votre agent exécute du bash

En cas de doute : demandez-lui d'expliquer ce qu'il va faire avant de le faire.

🔄 Gardez pi-mom à jour (fork Les Fadas)

Ce n'est **pas** `npm install -g @mariozechner/pi-mom` (paquet upstream). Sur le VPS : **relancer le script 2** (même `curl` que l'atelier), puis `systemctl --user restart pi-mom` — voir `docs/UPDATE.md` sur le repo workshop.

Bien utiliser son agent

Ce qui marche

✅ **Soyez précis** *"Crée un fichier recap.md avec les 5 points clés de notre réunion"*

✅ **Donnez du contexte** *"Je suis graphiste, mon client est dans la restauration…"*

✅ **Demandez-lui de s'expliquer** *"Avant de faire X, dis-moi ce que tu vas faire"*

✅ **Construisez des routines** *"Chaque matin à 9h, envoie-moi un résumé des emails importants"*

Ce qu'il faut éviter

❌ Les demandes vagues

❌ Confiance aveugle sur des actions irréversibles

❌ Oublier les limites de tokens de votre abonnement

❌ Ne jamais lui donner de contexte sur vous

Première mission ensemble

Interviewe-moi pour créer mon SYSTEM.md.
Pose-moi des questions **une par une** sur :

- Ta personnalité et ton ton
- Tes domaines d’expertise
- Ce que tu aimes et ce que tu n’aimes pas faire
- Ton prénom ou surnom
- Tes règles de base

Quand tu as assez d’infos, écris le fichier `~/agent-workspace/SYSTEM.md` et dis-moi de te redémarrer.

Après l’atelier

Missions — la suite sur ton VPS

- **Personnaliser** `SYSTEM.md` avec ton agent (Slack), puis `systemctl --user restart pi-mom`.
- **GitHub comme nuage** : pousser `~/agent-workspace` en **SSH** — sauvegarde, historique, accès depuis n'importe où. Étapes détaillées : `docs/MISSIONS.md` du repo workshop.
- **Quand tout ça tourne** → lance ton **premier projet agentique** : une mission claire dans Slack, travail sous `projects/<nom>/`, commits locaux puis `git push`.

💡 Les **messages vocaux Slack** peuvent être **transcrits** par Slack selon ton workspace — inutile d'empiler un autre outil pour capturer une idée avant de la donner à l'agent.

Mises à jour sur le VPS

Code `pi-mom` **(fork)** : derniers changements sur `github.com/clawassistantgf-dev/pi-mono` → sur le serveur, **relancez le script 2** — il resynchronise `~/pi-mono-fadas`, rebuild et réinstalle le CLI. Puis `systemctl --user restart pi-mom`.

Scripts atelier uniquement (`01` / `02` / `03`) : même idée, `curl` depuis `raw.githubusercontent.com/…/workshop-pi-mom/main/` puis exécution.

Guide complet : `docs/UPDATE.md` dans le dépôt workshop — pas besoin de publier sur le registre npm pour suivre notre fork.

```
curl -fsSL https://raw.githubusercontent.com/clawassistantgf-dev/workshop-pi-mom/main/02-install.sh -o /tmp/ws-pi-mom-02-insta
```

🚨 Si ça plante — les 2 pièges connus

⚠️ Script 1 — erreur "dpkg was interrupted"

Image Hostinger parfois livrée avec dpkg en attente. Fix en root :

```
dpkg --configure -a
deluser --remove-home AGENT 2>/dev/null
rm -f /etc/sudoers.d/90-fadas-agent
```

Puis relancez le script 1.

⚠️ Script 3 — service pi-mom inactif

Vérifiez les logs et la config ; causes fréquentes : `.keys` manquant, `~/.pi/mom/auth.json`, ou lingering désactivé après déconnexion SSH.

```
tail -40 ~/.pi-mom.log
systemctl --user status pi-mom
loginctl show-user $USER | grep Linger    # doit afficher Linger=yes
# sinon : sudo loginctl enable-linger $USER && systemctl --user restart pi-mom
```

C’est à vous 🫒

Votre agent tourne 24h/24 sur votre serveur.

github.com/clawassistantgf-dev/workshop-pi-mom — scripts + `docs/UPDATE.md` · `docs/MISSIONS.md`
github.com/clawassistantgf-dev/pi-mono — pi-mom version Les Fadas
github.com/badlogic/pi-mono — pi-mono original (Mario Zechner)

Le workspace Slack **Les Fadas de l'IA** reste actif.
@LeJ est là si vous avez des questions.